

Lecture 26

Dynamic Programming: Counting Boolean Parenthesizations, Bellman-Ford

Counting Boolean Parenthesizations

Counting Boolean Parenthesizations

Boolean operators: "and", "or", "xor".

Counting Boolean Parenthesizations

Boolean operators: "and", "or", "xor".



$F \text{ and } F = F$

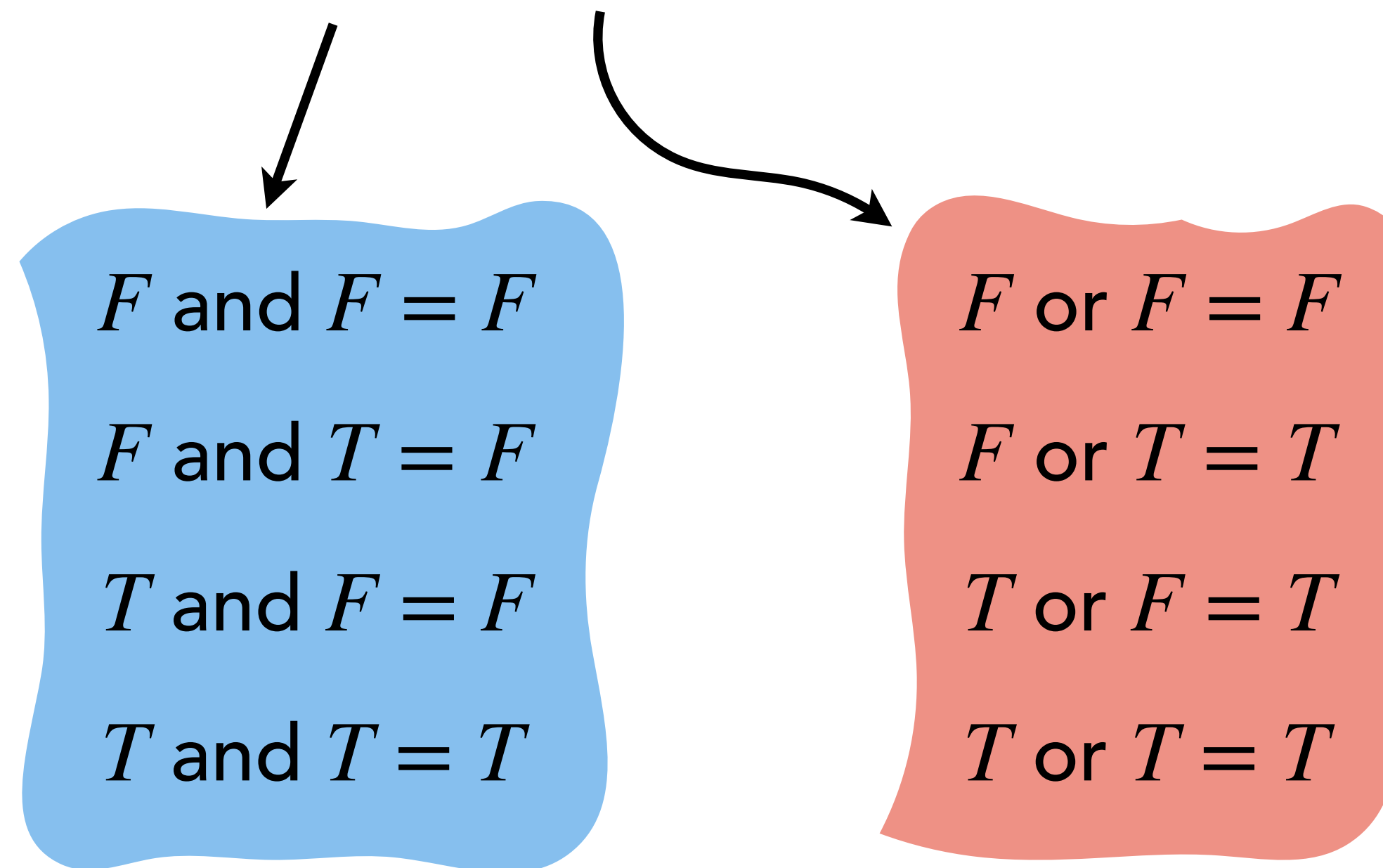
$F \text{ and } T = F$

$T \text{ and } F = F$

$T \text{ and } T = T$

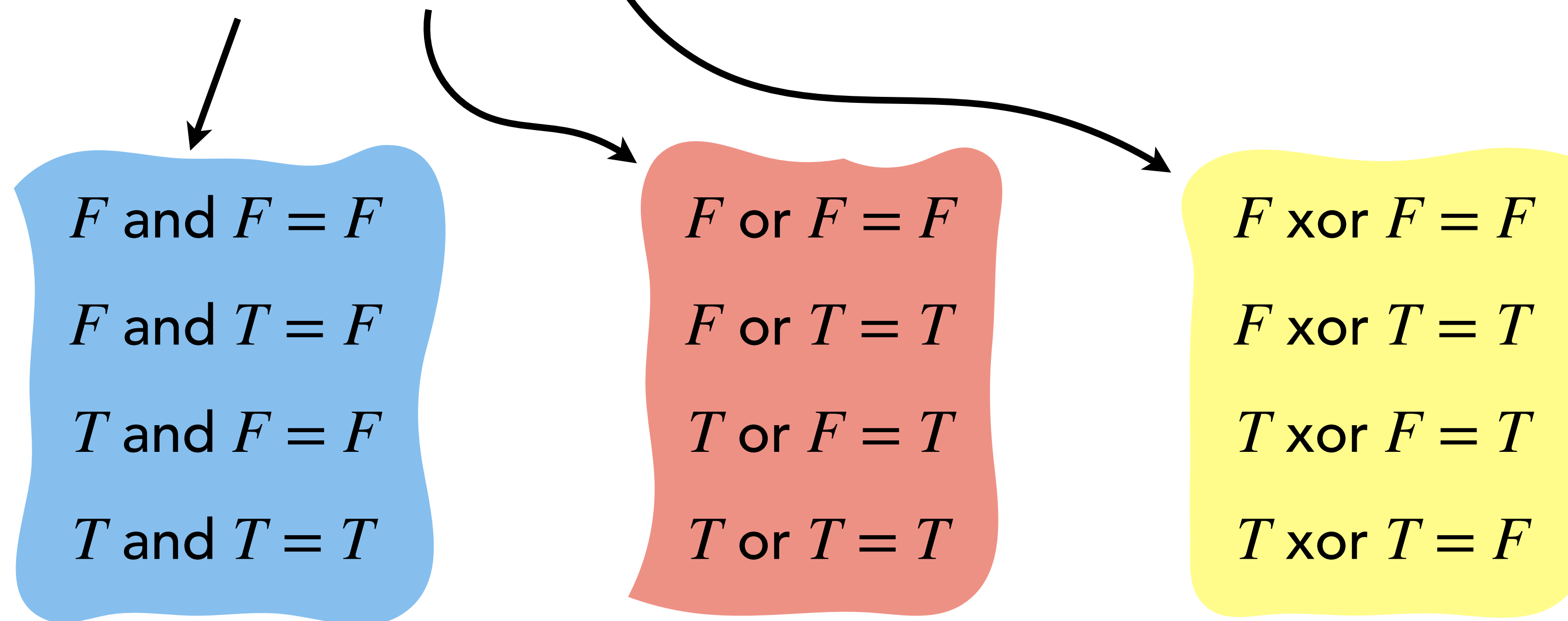
Counting Boolean Parenthesizations

Boolean operators: "and", "or", "xor".



Counting Boolean Parenthesizations

Boolean operators: "and", "or", "xor".



Counting Boolean Parenthesizations

Counting Boolean Parenthesizations

CBP:

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* − 1],

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* − 1],
where *vals* contains values *T*, *F*

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* − 1], where *vals* contains values *T*, *F* and *ops* contains operators "and", "or", "xor".

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* - 1], where *vals* contains values *T*, *F* and *ops* contains operators "and", "or", "xor".

Output: The number of ways given boolean expression can be parenthesized so that it

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* - 1], where *vals* contains values *T*, *F* and *ops* contains operators "and", "or", "xor".

Output: The **number of ways** given boolean expression can be **parenthesized** so that it evaluates to *T*.

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* − 1], where *vals* contains values *T*, *F* and *ops* contains operators "and", "or", "xor".

Output: The *number of ways* given boolean expression can be *parenthesized* so that it evaluates to *T*.

Example:

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* − 1], where *vals* contains values *T*, *F* and *ops* contains operators "and", "or", "xor".

Output: The *number of ways* given boolean expression can be *parenthesized* so that it evaluates to *T*.

Example: "*T* or *T* and *F* xor *T*" can be parenthesized in 4 ways to evaluate to *T*:

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* − 1], where *vals* contains values *T*, *F* and *ops* contains operators "and", "or", "xor".

Output: The *number of ways* given boolean expression can be *parenthesized* so that it evaluates to *T*.

Example: "*T* or *T* and *F* xor *T*" can be parenthesized in 4 ways to evaluate to *T*:

$$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$$

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* − 1], where *vals* contains values *T*, *F* and *ops* contains operators "and", "or", "xor".

Output: The **number of ways** given boolean expression can be **parenthesized** so that it evaluates to *T*.

Example: "*T* or *T* and *F* xor *T*" can be parenthesized in 4 ways to evaluate to *T*:

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

$(T \text{ or } (T \text{ and } (F \text{ xor } T)))$

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* − 1], where *vals* contains values *T*, *F* and *ops* contains operators "and", "or", "xor".

Output: The *number of ways* given boolean expression can be *parenthesized* so that it evaluates to *T*.

Example: "*T* or *T* and *F* xor *T*" can be parenthesized in 4 ways to evaluate to *T*:

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

$(T \text{ or } (T \text{ and } (F \text{ xor } T)))$

$((T \text{ or } T) \text{ and } (F \text{ xor } T))$

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* − 1], where *vals* contains values *T*, *F* and *ops* contains operators "and", "or", "xor".

Output: The **number of ways** given boolean expression can be **parenthesized** so that it evaluates to *T*.

Example: "*T* or *T* and *F* xor *T*" can be parenthesized in 4 ways to evaluate to *T*:

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

$(T \text{ or } (T \text{ and } (F \text{ xor } T)))$

$((T \text{ or } T) \text{ and } (F \text{ xor } T))$

$((((T \text{ or } T) \text{ and } F) \text{ xor } T))$

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays *vals*[1 : *n*] and *ops*[1 : *n* − 1], where *vals* contains values *T*, *F* and *ops* contains operators "and", "or", "xor".

Output: The *number of ways* given boolean expression can be *parenthesized* so that it evaluates to *T*.

Example: "*T* or *T* and *F* xor *T*" can be parenthesized in only one way to evaluate to *F*.

$$((T \text{ or } (T \text{ and } F)) \text{ xor } T)$$

What's a Parenthesized Boolean Expression?

What's a Parenthesized Boolean Expression?

Defn:

What's a Parenthesized Boolean Expression?

Defn: 1. T, F are parenthesized boolean expressions.

What's a Parenthesized Boolean Expression?

Defn: 1. T, F are parenthesized boolean expressions.
2. If α and β are two parenthesized boolean expressions,

What's a Parenthesized Boolean Expression?

Defn: 1. T, F are parenthesized boolean expressions.

2. If α and β are two parenthesized boolean expressions, then $(\alpha \text{ and } \beta)$, $(\alpha \text{ or } \beta)$, and

What's a Parenthesized Boolean Expression?

Defn: 1. T, F are parenthesized boolean expressions.

2. If α and β are two parenthesized boolean expressions, then $(\alpha \text{ and } \beta)$, $(\alpha \text{ or } \beta)$, and $(\alpha \text{ xor } \beta)$ are also parenthesized boolean expressions.

What's a Parenthesized Boolean Expression?

Defn: 1. T, F are parenthesized boolean expressions.

2. If α and β are two parenthesized boolean expressions, then $(\alpha \text{ and } \beta)$, $(\alpha \text{ or } \beta)$, and $(\alpha \text{ xor } \beta)$ are also parenthesized boolean expressions.

Example:

What's a Parenthesized Boolean Expression?

Defn: 1. T, F are parenthesized boolean expressions.

2. If α and β are two parenthesized boolean expressions, then $(\alpha \text{ and } \beta)$, $(\alpha \text{ or } \beta)$, and $(\alpha \text{ xor } \beta)$ are also parenthesized boolean expressions.

Example: $((T \text{ or } T) \text{ and } (F \text{ xor } T))$ is a parenthesized boolean expressions

What's a Parenthesized Boolean Expression?

Defn: 1. T, F are parenthesized boolean expressions.

2. If α and β are two parenthesized boolean expressions, then $(\alpha \text{ and } \beta)$, $(\alpha \text{ or } \beta)$, and $(\alpha \text{ xor } \beta)$ are also parenthesized boolean expressions.

Example: $((T \text{ or } T) \text{ and } (F \text{ xor } T))$ is a parenthesized boolean expressions

T

What's a Parenthesized Boolean Expression?

Defn: 1. T, F are parenthesized boolean expressions.

2. If α and β are two parenthesized boolean expressions, then $(\alpha \text{ and } \beta)$, $(\alpha \text{ or } \beta)$, and $(\alpha \text{ xor } \beta)$ are also parenthesized boolean expressions.

Example: $((T \text{ or } T) \text{ and } (F \text{ xor } T))$ is a parenthesized boolean expressions

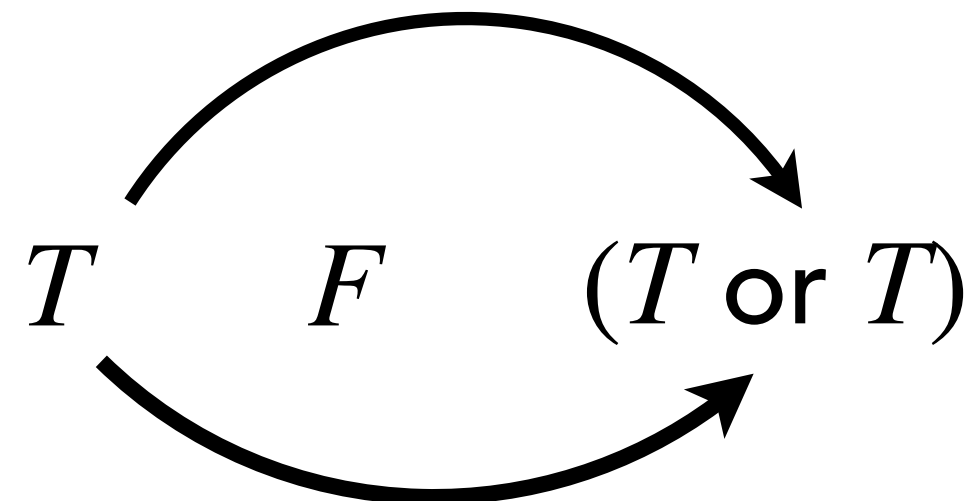
$T \quad F$

What's a Parenthesized Boolean Expression?

Defn: 1. T, F are **parenthesized boolean expressions**.

2. If α and β are two parenthesized boolean expressions, then $(\alpha \text{ and } \beta)$, $(\alpha \text{ or } \beta)$, and $(\alpha \text{ xor } \beta)$ are also **parenthesized boolean expressions**.

Example: $((T \text{ or } T) \text{ and } (F \text{ xor } T))$ is a parenthesized boolean expressions

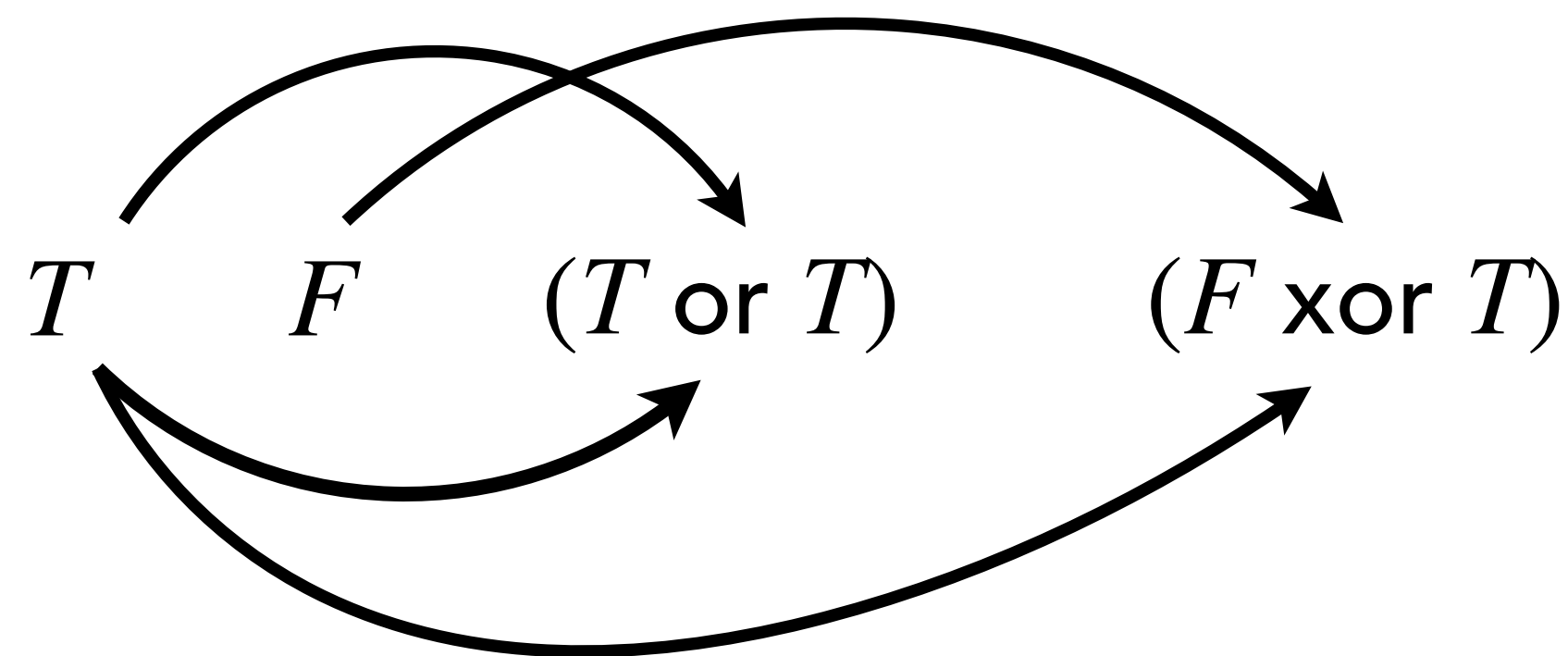


What's a Parenthesized Boolean Expression?

Defn: 1. T, F are **parenthesized boolean expressions**.

2. If α and β are two parenthesized boolean expressions, then $(\alpha \text{ and } \beta)$, $(\alpha \text{ or } \beta)$, and $(\alpha \text{ xor } \beta)$ are also **parenthesized boolean expressions**.

Example: $((T \text{ or } T) \text{ and } (F \text{ xor } T))$ is a parenthesized boolean expressions

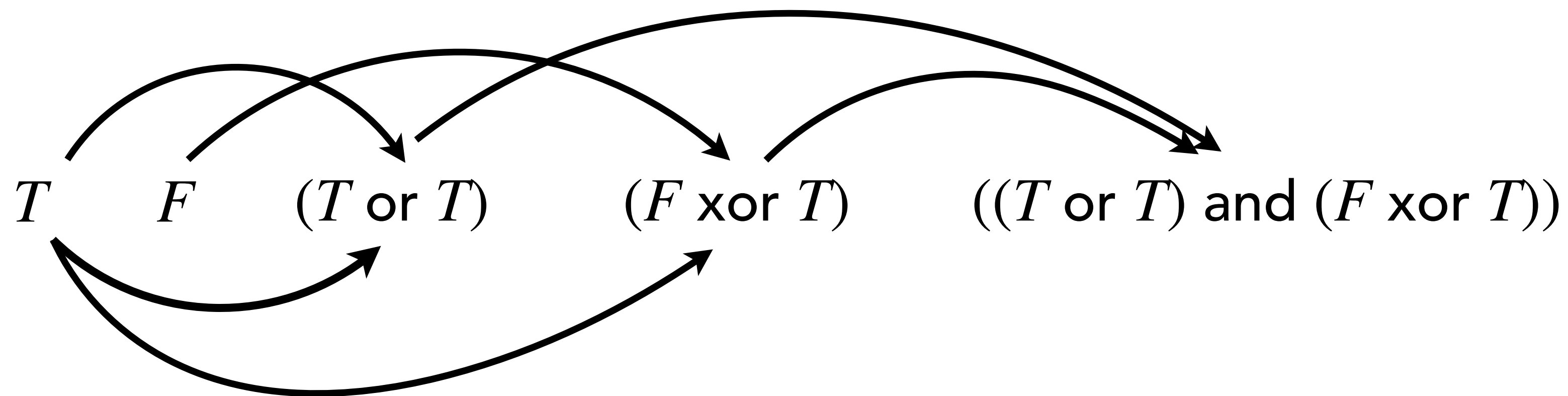


What's a Parenthesized Boolean Expression?

Defn: 1. T, F are **parenthesized boolean expressions**.

2. If α and β are two parenthesized boolean expressions, then $(\alpha \text{ and } \beta)$, $(\alpha \text{ or } \beta)$, and $(\alpha \text{ xor } \beta)$ are also **parenthesized boolean expressions**.

Example: $((T \text{ or } T) \text{ and } (F \text{ xor } T))$ is a parenthesized boolean expressions



Counting Substructure for CBP

Counting Substructure for CBP

Observation: A parenthesized expression is:

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .
- Formed from two parenthesized subexpressions.

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .
- Formed from two parenthesized subexpressions.

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .
- Formed from two parenthesized subexpressions.

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .
- Formed from two parenthesized subexpressions.

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

$(T \text{ or } (T \text{ and } (F \text{ xor } T)))$

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .
- Formed from two parenthesized subexpressions.

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

$(T \text{ or } (T \text{ and } (F \text{ xor } T)))$

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .
- Formed from two parenthesized subexpressions.

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

$(T \text{ or } (T \text{ and } (F \text{ xor } T)))$

$((T \text{ or } T) \text{ and } (F \text{ xor } T))$

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .
- Formed from two parenthesized subexpressions.

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

$(T \text{ or } (T \text{ and } (F \text{ xor } T)))$

$((T \text{ or } T) \text{ and } (F \text{ xor } T))$

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .
- Formed from two parenthesized subexpressions.

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

$(T \text{ or } (T \text{ and } (F \text{ xor } T)))$

$((T \text{ or } T) \text{ and } (F \text{ xor } T))$

$((((T \text{ or } T) \text{ and } F) \text{ xor } T)$

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .
- Formed from two parenthesized subexpressions.

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

$(T \text{ or } (T \text{ and } (F \text{ xor } T)))$

$((T \text{ or } T) \text{ and } (F \text{ xor } T))$

$((((T \text{ or } T) \text{ and } F) \text{ xor } T))$

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .
- Formed from two parenthesized subexpressions.

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

$(T \text{ or } (T \text{ and } (F \text{ xor } T)))$

$((T \text{ or } T) \text{ and } (F \text{ xor } T))$

$((((T \text{ or } T) \text{ and } F) \text{ xor } T))$

$((T \text{ or } (T \text{ and } F)) \text{ xor } T)$

Counting Substructure for CBP

Observation: A parenthesized expression is:

- T, F .
- Formed from two parenthesized subexpressions.

$(T \text{ or } ((T \text{ and } F) \text{ xor } T))$

$(T \text{ or } (T \text{ and } (F \text{ xor } T)))$

$((T \text{ or } T) \text{ and } (F \text{ xor } T))$

$((((T \text{ or } T) \text{ and } F) \text{ xor } T)$

$((T \text{ or } (T \text{ and } F)) \text{ xor } T)$

Counting Substructure for CBP

Parenthesizing " T or F and F xor T and T xor F or T " involves parenthesizing following sets of two subexpressions:

Counting Substructure for CBP

Parenthesizing " T or F and F xor T and T xor F or T " involves parenthesizing following sets of two subexpressions:

T or F and F xor T and T xor F or T

Counting Substructure for CBP

Parenthesizing " T or F and F xor T and T xor F or T " involves parenthesizing following sets of two subexpressions:

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

Counting Substructure for CBP

Parenthesizing " T or F and F xor T and T xor F or T " involves parenthesizing following sets of two subexpressions:

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

Counting Substructure for CBP

Parenthesizing " T or F and F xor T and T xor F or T " involves parenthesizing following sets of two subexpressions:

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

Counting Substructure for CBP

Parenthesizing " T or F and F xor T and T xor F or T " involves parenthesizing following sets of two subexpressions:

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

Counting Substructure for CBP

Parenthesizing " T or F and F xor T and T xor F or T " involves parenthesizing following sets of two subexpressions:

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

T or F and F xor T and T xor F or T

Counting Substructure for CBP

Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T ,

Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?

Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?

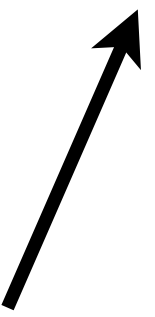
T or F and F xor T and T xor F or T

Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?

T or F and F xor T and T xor F or T

Number of ways to parenthesize
this subexpression so that it's T



Counting Substructure for CBP

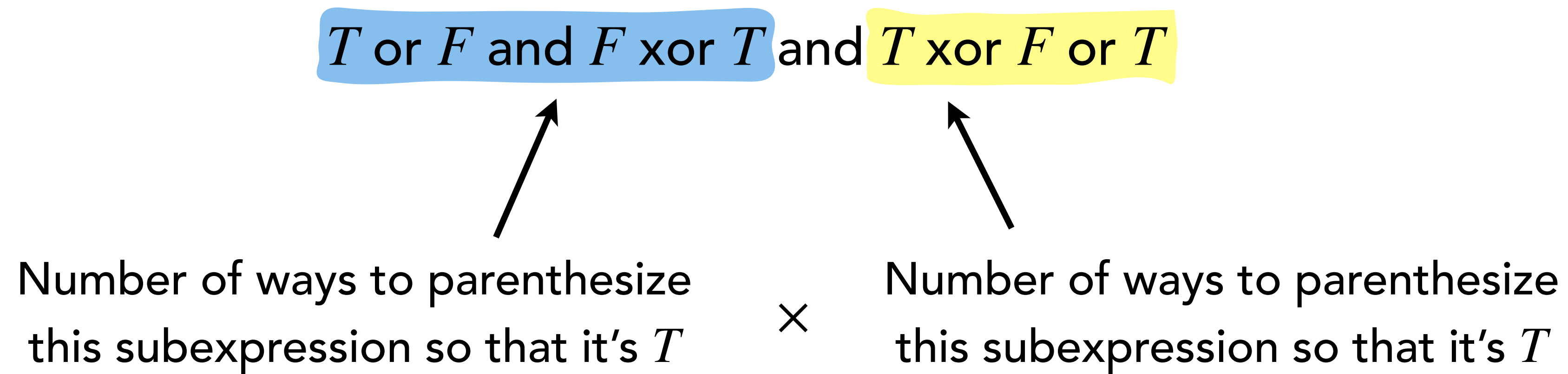
In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?

T or F and F xor T and T xor F or T

Number of ways to parenthesize
this subexpression so that it's T ×

Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?



Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?

Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?

T or F and F xor T and T xor F or T

Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?

T or F and F xor T and T xor F or T



Number of ways to parenthesize
this subexpression so that it's T

Counting Substructure for CBP

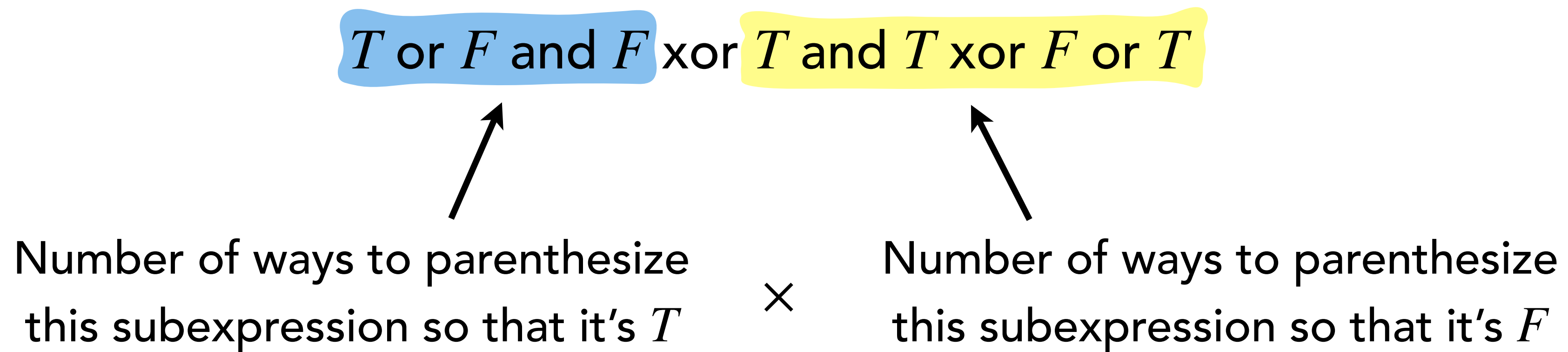
In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?

T or F and F xor T and T xor F or T

Number of ways to parenthesize
this subexpression so that it's T ×

Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?



Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?

Number of ways to parenthesize
this subexpression so that it's F

T or F and F xor T and T xor F or T

Number of ways to parenthesize
this subexpression so that it's T

$\times$

Number of ways to parenthesize
this subexpression so that it's F

Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?

Number of ways to parenthesize
this subexpression so that it's F

×

T or F and F xor T and T xor F or T

Number of ways to parenthesize
this subexpression so that it's T

×

Number of ways to parenthesize
this subexpression so that it's F

Counting Substructure for CBP

In how many ways " T or F and F xor T and T xor F or T " can be parenthesized to be T , if parenthesization involves parenthesizing the following subexpressions?

Number of ways to parenthesize
this subexpression so that it's F

×

Number of ways to parenthesize
this subexpression so that it's T

T or F and F xor T and T xor F or T

Number of ways to parenthesize
this subexpression so that it's T

×

Number of ways to parenthesize
this subexpression so that it's F

Counting Boolean Parenthesizations

CBP:

Input: Given a boolean expression in the form of two arrays $vals[1 : n]$ and $ops[1 : n - 1]$, where $vals$ contains values T, F and ops contains operators "and", "or", "xor".

Output: The **number of ways** given boolean expression can be **parenthesized** so that it evaluates to T .

Recurrence for CBP

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .



We are interested in $T(1, n)$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

Recurrence for CBP

Recurrence for CBP

Find $T(1,6)$ for $vals =$

T	F	F	T	T	F
-----	-----	-----	-----	-----	-----

 and $ops =$

or	and	xor	and	xor
----	-----	-----	-----	-----

 .

Recurrence for CBP

Find $T(1,6)$ for $vals =$

T	F	F	T	T	F
-----	-----	-----	-----	-----	-----

 and $ops =$

or	and	xor	and	xor
----	-----	-----	-----	-----

.

" T or F and F xor T and T xor F "

Recurrence for CBP

Find $T(1,6)$ for $vals =$

T	F	F	T	T	F
-----	-----	-----	-----	-----	-----

 and $ops =$

or	and	xor	and	xor
----	-----	-----	-----	-----

 .

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

Recurrence for CBP

Find $T(1,6)$ for $vals =$

T	F	F	T	T	F
-----	-----	-----	-----	-----	-----

 and $ops =$

or	and	xor	and	xor
----	-----	-----	-----	-----

 .

" T or F and F xor T and T xor F " $\rightarrow$

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

Recurrence for CBP

Find $T(1,6)$ for $vals =$

T	F	F	T	T	F
-----	-----	-----	-----	-----	-----

 and $ops =$

or	and	xor	and	xor
----	-----	-----	-----	-----

 .

" T or F and F xor T and T xor F " $\longrightarrow T(1,1) \cdot T(2,6) + T(1,1) \cdot F(2,6) + F(1,1) \cdot T(2,6)$

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

Recurrence for CBP

Find $T(1,6)$ for $vals =$

T	F	F	T	T	F
-----	-----	-----	-----	-----	-----

 and $ops =$

or	and	xor	and	xor
----	-----	-----	-----	-----

 .

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F" \rightarrow T(1,1) \cdot T(2,6) + T(1,1) \cdot F(2,6) + F(1,1) \cdot T(2,6)$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F"$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F"$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F"$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F"$$

$\because ops[1] = \text{or}$

Recurrence for CBP

Find $T(1,6)$ for $vals =$

T	F	F	T	T	F
-----	-----	-----	-----	-----	-----

 and $ops =$

or	and	xor	and	xor
----	-----	-----	-----	-----

 .

" T or F and F xor T and T xor F " $\rightarrow T(1,1) \cdot T(2,6) + T(1,1) \cdot F(2,6) + F(1,1) \cdot T(2,6)$

" T or F and F xor T and T xor F " $\rightarrow$ $\quad\quad\quad +$
 $\quad\quad\quad T(1,2) \cdot T(3,6)$

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

Recurrence for CBP

Find $T(1,6)$ for $vals =$

T	F	F	T	T	F
-----	-----	-----	-----	-----	-----

 and $ops =$

or	and	xor	and	xor
----	-----	-----	-----	-----

 .

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F" \longrightarrow T(1,1) \cdot T(2,6) + T(1,1) \cdot F(2,6) + F(1,1) \cdot T(2,6)$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F" \longrightarrow$$

$$+ \underbrace{T(1,2) \cdot T(3,6)}$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F"$$

$$\because ops[2] = \text{and}$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F"$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F"$$

Recurrence for CBP

Find $T(1,6)$ for $vals =$

T	F	F	T	T	F
-----	-----	-----	-----	-----	-----

 and $ops =$

or	and	xor	and	xor
----	-----	-----	-----	-----

 .

" T or F and F xor T and T xor F " $\rightarrow T(1,1) \cdot T(2,6) + T(1,1) \cdot F(2,6) + F(1,1) \cdot T(2,6)$

" T or F and F xor T and T xor F " $\rightarrow$ $\quad\quad\quad +$
 $\quad\quad\quad T(1,2) \cdot T(3,6)$

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

" T or F and F xor T and T xor F "

Recurrence for CBP

Find $T(1,6)$ for $vals =$

T	F	F	T	T	F
-----	-----	-----	-----	-----	-----

 and $ops =$

or	and	xor	and	xor
----	-----	-----	-----	-----

 .

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F" \longrightarrow T(1,1) \cdot T(2,6) + T(1,1) \cdot F(2,6) + F(1,1) \cdot T(2,6)$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F" \longrightarrow \begin{matrix} + \\ T(1,2) \cdot T(3,6) \end{matrix}$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F" \longrightarrow \begin{matrix} + \\ T(1,3) \cdot F(4,6) + F(1,3) \cdot T(4,6) \end{matrix}$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F"$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F"$$

Recurrence for CBP

Find $T(1,6)$ for $vals =$

T	F	F	T	T	F
-----	-----	-----	-----	-----	-----

 and $ops =$

or	and	xor	and	xor
----	-----	-----	-----	-----

 .

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F" \longrightarrow T(1,1) \cdot T(2,6) + T(1,1) \cdot F(2,6) + F(1,1) \cdot T(2,6)$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F" \longrightarrow \begin{matrix} + \\ T(1,2) \cdot T(3,6) \end{matrix}$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F" \longrightarrow \begin{matrix} + \\ T(1,3) \cdot F(4,6) + F(1,3) \cdot T(4,6) \end{matrix}$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F" \longrightarrow \begin{matrix} + \\ T(1,4) \cdot T(5,6) \end{matrix}$$

$$"T \text{ or } F \text{ and } F \text{ xor } T \text{ and } T \text{ xor } F"$$

Recurrence for CBP

Find $T(1,6)$ for $vals = \begin{bmatrix} T & F & F & T & T & F \end{bmatrix}$ and $ops = \begin{bmatrix} \text{or} & \text{and} & \text{xor} & \text{and} & \text{xor} \end{bmatrix}$.

"T or F and F xor T and T xor F"	→	$T(1,1) \cdot T(2,6) + T(1,1) \cdot F(2,6) + F(1,1) \cdot T(2,6)$
		+
"T or F and F xor T and T xor F"	→	$T(1,2) \cdot T(3,6)$
		+
"T or F and F xor T and T xor F"	→	$T(1,3) \cdot F(4,6) + F(1,3) \cdot T(4,6)$
		+
"T or F and F xor T and T xor F"	→	$T(1,4) \cdot T(5,6)$
		+
"T or F and F xor T and T xor F"	→	$T(1,5) \cdot F(6,6) + F(1,5) \cdot T(6,6)$

Recurrence for CBP

Find $T(1,6)$ for $vals = \begin{bmatrix} T & F & F & T & T & F \end{bmatrix}$ and $ops = \begin{bmatrix} \text{or} & \text{and} & \text{xor} & \text{and} & \text{xor} \end{bmatrix}$.

$$\begin{aligned}
& \text{"T or F and F xor T and T xor F"} &\longrightarrow& T(1,1) \cdot T(2,6) + T(1,1) \cdot F(2,6) + F(1,1) \cdot T(2,6) \\
& &&+ \\
& \text{"T or F and F xor T and T xor F"} &\longrightarrow& T(1,2) \cdot T(3,6) \\
& &&+ \\
& \text{"T or F and F xor T and T xor F"} &\longrightarrow& T(1,3) \cdot F(4,6) + F(1,3) \cdot T(4,6) \\
& &&+ \\
& \text{"T or F and F xor T and T xor F"} &\longrightarrow& T(1,4) \cdot T(5,6) \\
& &&+ \\
& \text{"T or F and F xor T and T xor F"} &\longrightarrow& T(1,5) \cdot F(6,6) + F(1,5) \cdot T(6,6) \\
& &&= \\
& && T(1,6)
\end{aligned}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$T(i, j)$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) =$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \left\{ \right.$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \left\{ \begin{array}{l} , \quad \text{if } i = j \text{ and } vals[i] = T \end{array} \right.$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \begin{cases} 1, & \text{if } i = j \text{ and } \text{vals}[i] = T \\ 1, & \text{if } i = j \text{ and } \text{vals}[i] = F \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \begin{cases} 1, & \text{if } i = j \text{ and } vals[i] = T \\ , & \text{if } i = j \text{ and } vals[i] = F \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \begin{cases} 1, & \text{if } i = j \text{ and } vals[i] = T \\ 0, & \text{if } i = j \text{ and } vals[i] = F \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \begin{cases} 1, & \text{if } i = j \text{ and } vals[i] = T \\ 0, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} & \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \begin{cases} 1, & \text{if } i = j \text{ and } vals[i] = T \\ 0, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \begin{cases} 1, & \text{if } i = j \text{ and } vals[i] = T \\ 0, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} T(i, k) \cdot F(k+1, j), & \text{if } ops[k] = \text{"and"} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \begin{cases} 1, & \text{if } i = j \text{ and } vals[i] = T \\ 0, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \begin{cases} T(i, k) * F(k+1, j), & \text{if } ops[k] = \text{"and"} \\ F(i, k) * T(k+1, j), & \text{if } ops[k] = \text{"xor"} \end{cases} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \begin{cases} 1, & \text{if } i = j \text{ and } vals[i] = T \\ 0, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \begin{cases} T(i, k) * F(k+1, j), & \text{if } ops[k] = \text{"and"} \\ F(i, k) * T(k+1, j), & \text{if } ops[k] = \text{"xor"} \\ T(i, k) * T(k+1, j), & \text{if } ops[k] = \text{"or"} \end{cases} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \begin{cases} 1, & \text{if } i = j \text{ and } vals[i] = T \\ 0, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \begin{cases} T[i, k] \cdot T[k+1, j], & \text{if } ops[k] = \text{"and"} \\ , & \text{if } ops[k] = \text{"xor"} \\ , & \text{if } ops[k] = \text{"or"} \end{cases} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \begin{cases} 1, & \text{if } i = j \text{ and } vals[i] = T \\ 0, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \begin{cases} T[i, k] \cdot T[k+1][j], & \text{if } ops[k] = \text{"and"} \\ T[i, k] \cdot F[k+1][j] + F[i, k] \cdot T[k+1][j], & \text{if } ops[k] = \text{"xor"} \\ , & \text{if } ops[k] = \text{"or"} \end{cases} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$T(i, j) = \begin{cases} 1, & \text{if } i = j \text{ and } vals[i] = T \\ 0, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \begin{cases} T[i, k] \cdot T[k+1][j], & \text{if } ops[k] = \text{"and"} \\ T[i, k] \cdot F[k+1][j] + F[i, k] \cdot T[k+1][j], & \text{if } ops[k] = \text{"xor"} \\ T[i, k] \cdot T[k+1][j] + T[i, k] \cdot F[k+1][j] + F[i, k] \cdot T[k+1][j], & \text{if } ops[k] = \text{"or"} \end{cases} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$F(i, j)$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) =$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \left\{ \right.$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \left\{ \begin{array}{l} , \quad \text{if } i = j \text{ and } vals[i] = T \end{array} \right.$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \begin{cases} , & \text{if } i = j \text{ and } vals[i] = T \\ , & \text{if } i = j \text{ and } vals[i] = F \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \begin{cases} 0, & \text{if } i = j \text{ and } \text{vals}[i] = T \\ , & \text{if } i = j \text{ and } \text{vals}[i] = F \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \begin{cases} 0, & \text{if } i = j \text{ and } \text{vals}[i] = T \\ 1, & \text{if } i = j \text{ and } \text{vals}[i] = F \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \begin{cases} 0, & \text{if } i = j \text{ and } vals[i] = T \\ 1, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \begin{cases} 0, & \text{if } i = j \text{ and } vals[i] = T \\ 1, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \begin{cases} 0, & \text{if } i = j \text{ and } vals[i] = T \\ 1, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \left(F(i, k) \cdot T(k+1, j) + T(i, k) \cdot F(k+1, j) \right), & \text{if } ops[k] = \text{"or"} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \begin{cases} 0, & \text{if } i = j \text{ and } vals[i] = T \\ 1, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \left(F(i, k) \cdot T(k+1, j) + T(i, k) \cdot F(k+1, j) \right), & \text{if } ops[k] = \text{"or"} \\ \sum_{i \leq k \leq j-1} \left(F(i, k) \cdot F(k+1, j) + T(i, k) \cdot T(k+1, j) \right), & \text{if } ops[k] = \text{"xor"} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \begin{cases} 0, & \text{if } i = j \text{ and } vals[i] = T \\ 1, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \left(\begin{aligned} & F(i, k) * T(k+1, j) + T(i, k) * F(k+1, j), & \text{if } ops[k] = \text{"or"} \\ & F(i, k) * F(k+1, j) + T(i, k) * T(k+1, j), & \text{if } ops[k] = \text{"xor"} \\ & F(i, k) * F(k+1, j) + F(i, k) * T(k+1, j) + T(i, k) * F(k+1, j), & \text{if } ops[k] = \text{"and"} \end{aligned} \right) \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \begin{cases} 0, & \text{if } i = j \text{ and } vals[i] = T \\ 1, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \left\{ \begin{array}{l} F[i, k] \cdot F[k+1, j], \quad \text{if } ops[k] = \text{"or"} \\ , \quad \text{if } ops[k] = \text{"xor"} \\ , \quad \text{if } ops[k] = \text{"and"} \end{array} \right. \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \begin{cases} 0, & \text{if } i = j \text{ and } vals[i] = T \\ 1, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \begin{cases} F[i, k] \cdot F[k+1][j], & \text{if } ops[k] = \text{"or"} \\ T[i, k] \cdot T[k+1][j] + F[i, k] \cdot F[k+1][j], & \text{if } ops[k] = \text{"xor"} \\ , & \text{if } ops[k] = \text{"and"} \end{cases} \end{cases}$$

Recurrence for CBP

Let $T(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is T .

Let $F(i, j)$ = # ways to parenthesize subexpression from i th to j th symbol such that it is F .

$$F(i, j) = \begin{cases} 0, & \text{if } i = j \text{ and } vals[i] = T \\ 1, & \text{if } i = j \text{ and } vals[i] = F \\ \sum_{i \leq k \leq j-1} \begin{cases} F[i, k] \cdot F[k+1][j], & \text{if } ops[k] = \text{"or"} \\ T[i, k] \cdot T[k+1][j] + F[i, k] \cdot F[k+1][j], & \text{if } ops[k] = \text{"xor"} \\ F[i, k] \cdot F[k+1][j] + T[i, k] \cdot F[k+1][j] + F[i, k] \cdot T[k+1][j], & \text{if } ops[k] = \text{"and"} \end{cases} \end{cases}$$

Single-Source Shortest Path Problem

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

SSSP:

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

SSSP:

Input: A weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$ with **no negative**

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

SSSP:

Input: A weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$ with **no negative weight cycles** and a vertex s .

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

SSSP:

Input: A weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$ with **no negative weight cycles** and a vertex s .

Output: Compute distance from s to every other vertex. That is, an array $d[1 : |V|]$,

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

SSSP:

Input: A weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$ with **no negative weight cycles** and a vertex s .

Output: Compute distance from s to every other vertex. That is, an array $d[1 : |V|]$, such that $d[i] = \delta(s, i)$.

Single-Source Shortest Path Problem

Let $\delta(s, v)$ denote the weight of a **shortest path** from s to v .

If v is not reachable (or connected) from s , then $\delta(s, v) = \infty$.

A polytime solution for *SSSP* when graph has negative weight cycle is unlikely to exist.
We will see why in future.



SSSP:

Input: A weighted graph $G = (V, E)$ with weight function $w : E \rightarrow \mathbb{R}$ with **no negative weight cycles** and a vertex s .

Output: Compute distance from s to every other vertex. That is, an array $d[1 : |V|]$, such that $d[i] = \delta(s, i)$.

Optimal Subpath Property

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof:

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof: Let P be a shortest path from s to v of weight w .

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

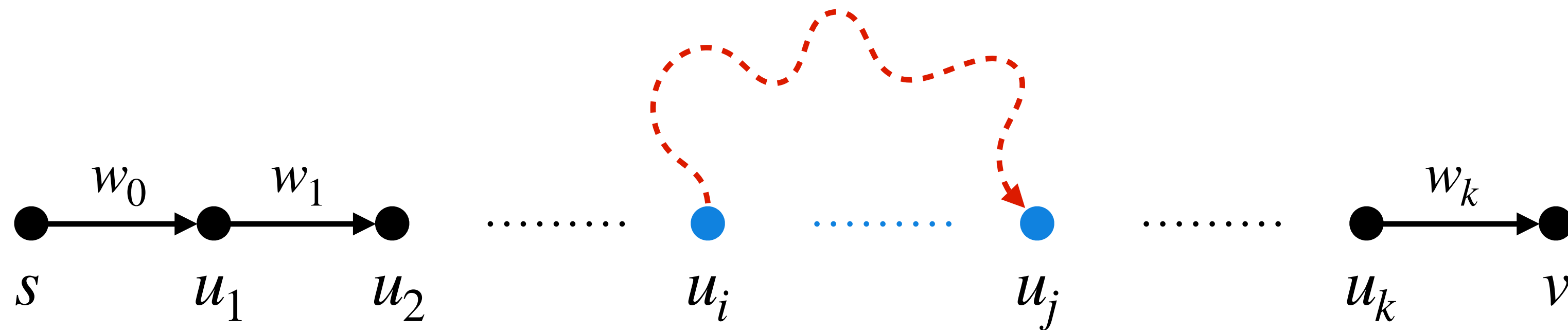
Proof: Let P be a shortest path from s to v of weight w .



Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof: Let P be a shortest path from s to v of weight w .

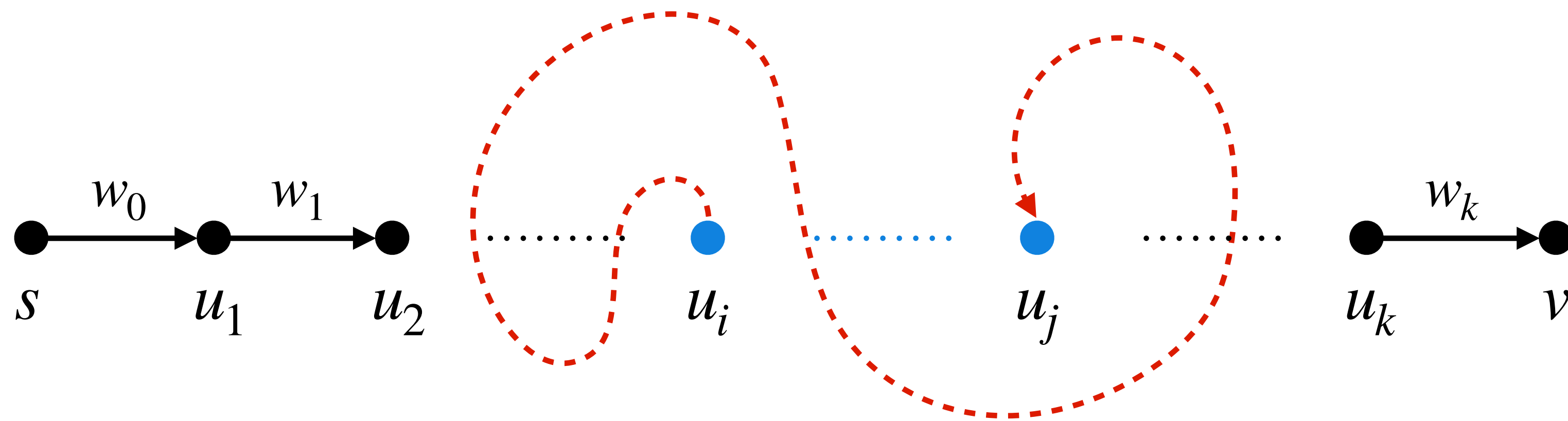


Suppose $\exists$ a shorter path from u_i to u_j .

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof: Let P be a shortest path from s to v of weight w .

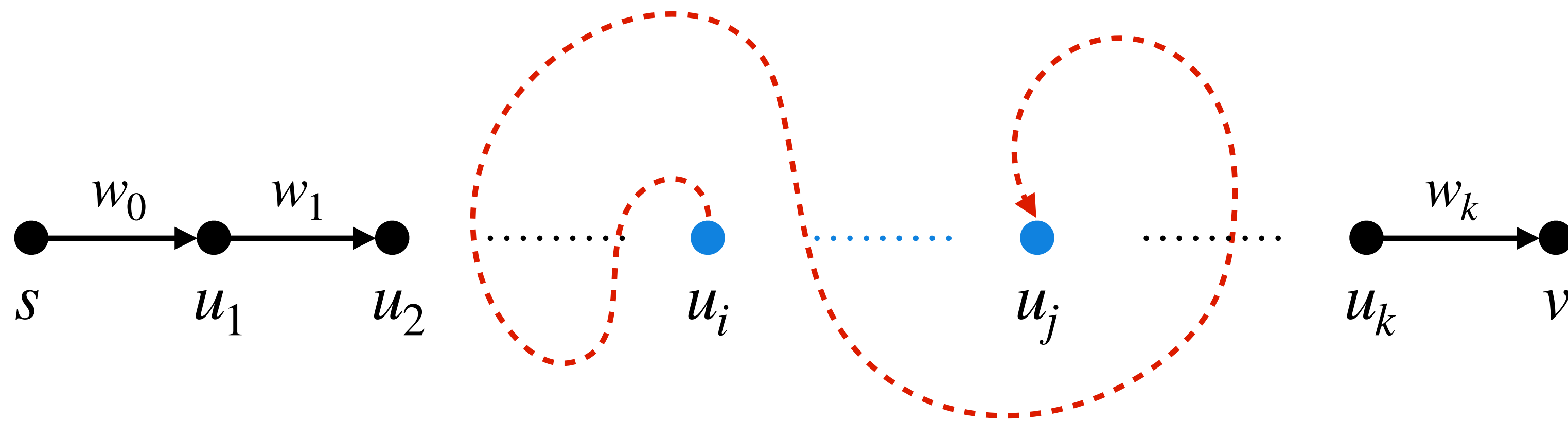


Suppose $\exists$ a shorter path from u_i to u_j .

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof: Let P be a shortest path from s to v of weight w .



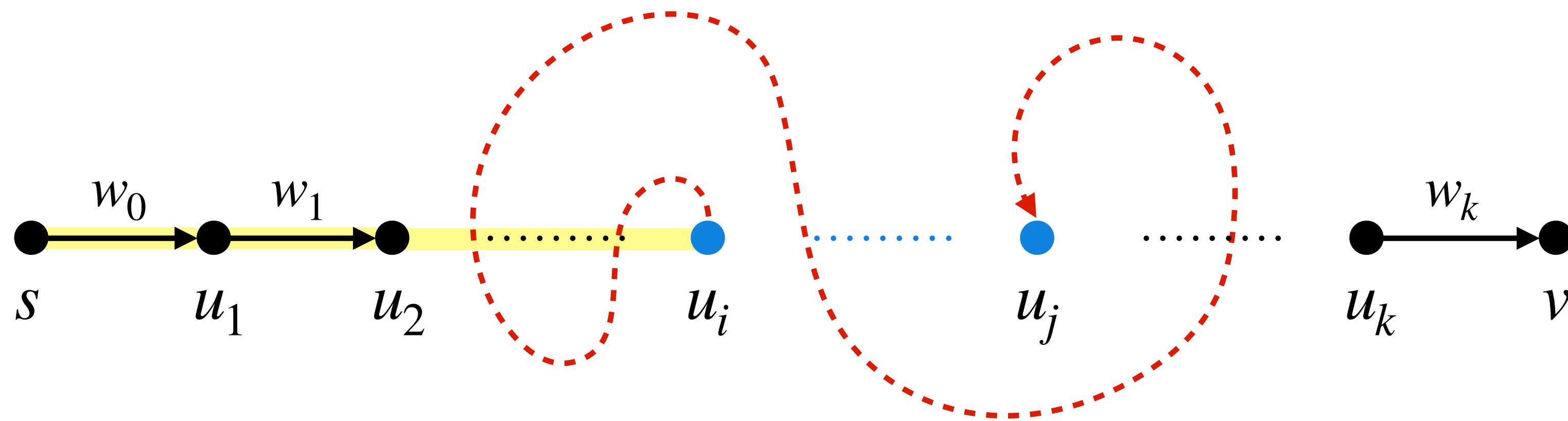
Suppose $\exists$ a shorter path from u_i to u_j .

Then, $\exists$ a walk P' from s to v with weight $< w$.

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof: Let P be a shortest path from s to v of weight w .



Suppose $\exists$ a shorter path from u_i to u_j .

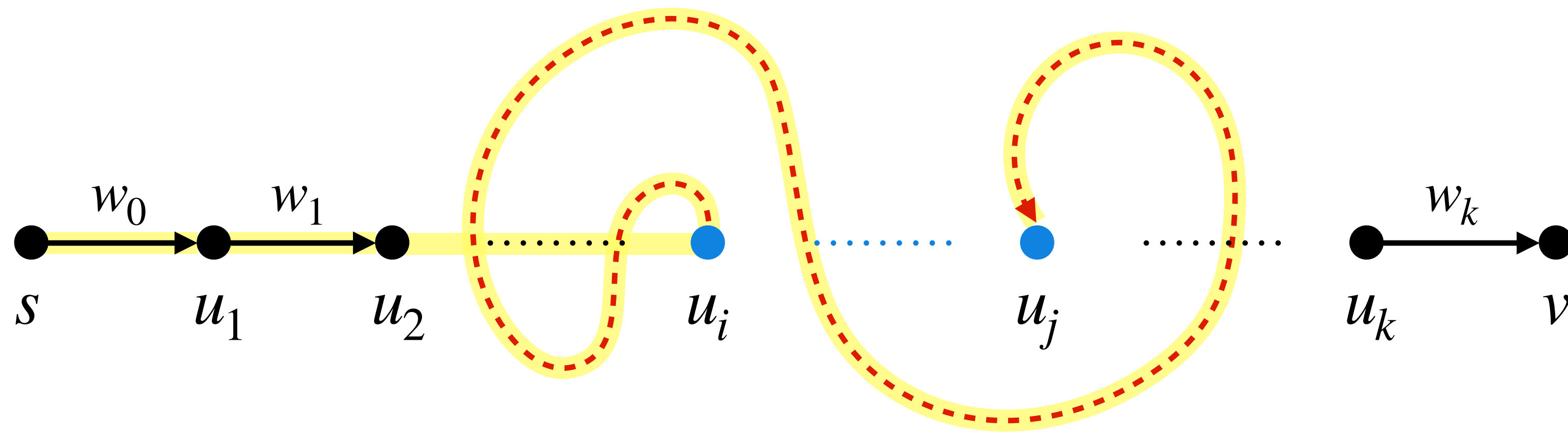
Then, $\exists$ a walk P' from s to v with weight $< w$.

$P' =$ path from s to u_i

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof: Let P be a shortest path from s to v of weight w .



Suppose $\exists$ a shorter path from u_i to u_j .

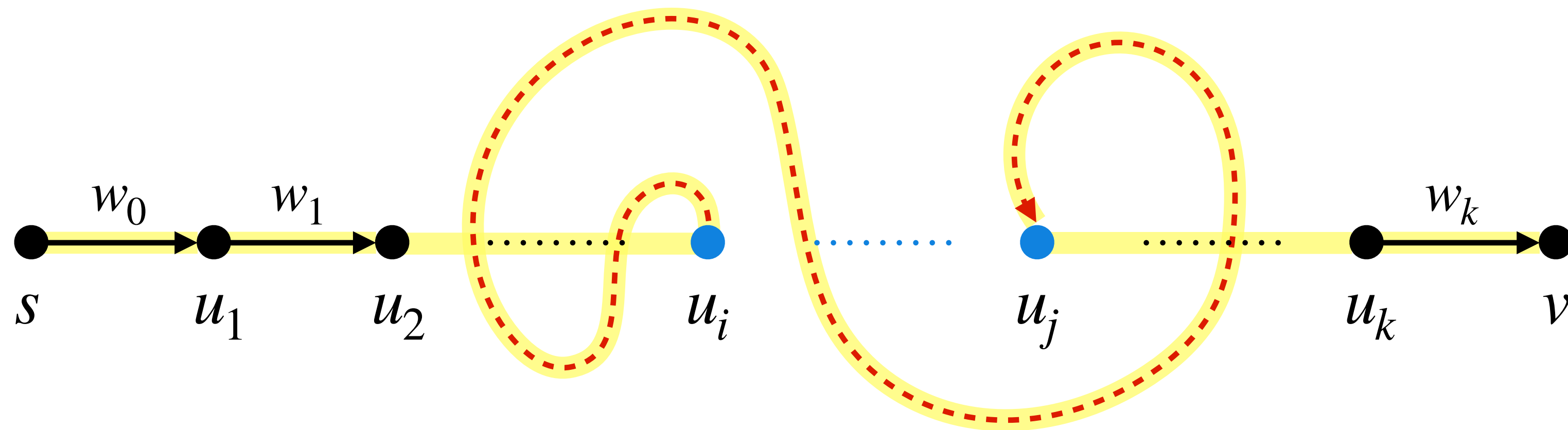
Then, $\exists$ a walk P' from s to v with weight $< w$.

$$P' = \text{path from } s \text{ to } u_i + \text{shorter path from } u_i \text{ to } u_j$$

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof: Let P be a shortest path from s to v of weight w .



Suppose $\exists$ a shorter path from u_i to u_j .

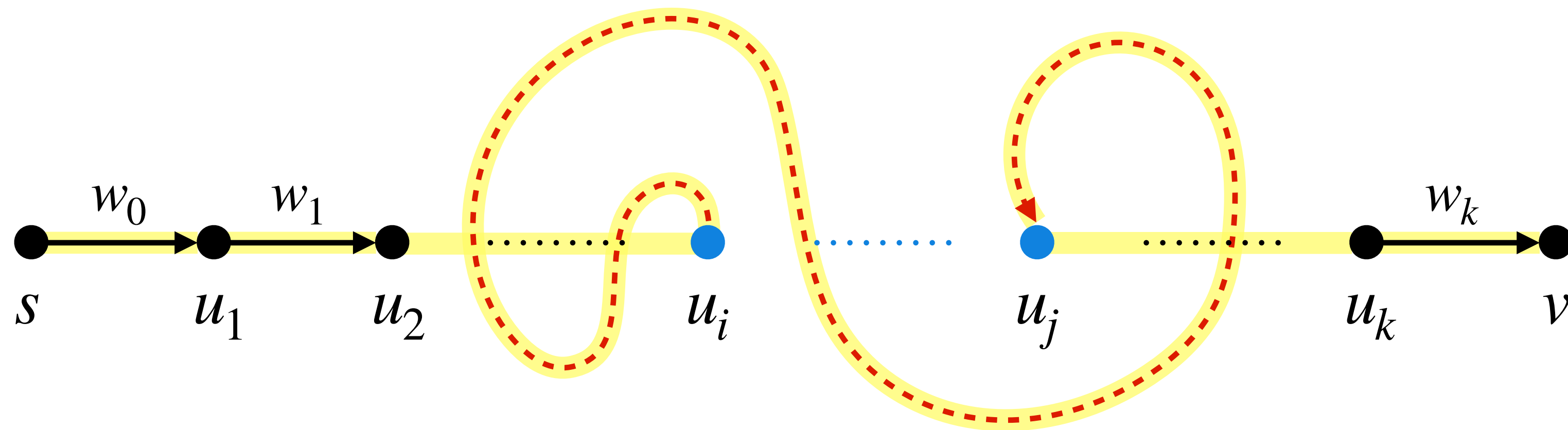
Then, $\exists$ a walk P' from s to v with weight $< w$.

$$P' = \text{path from } s \text{ to } u_i + \text{shorter path from } u_i \text{ to } u_j + \text{path from } u_j \text{ to } v$$

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof: Let P be a shortest path from s to v of weight w .



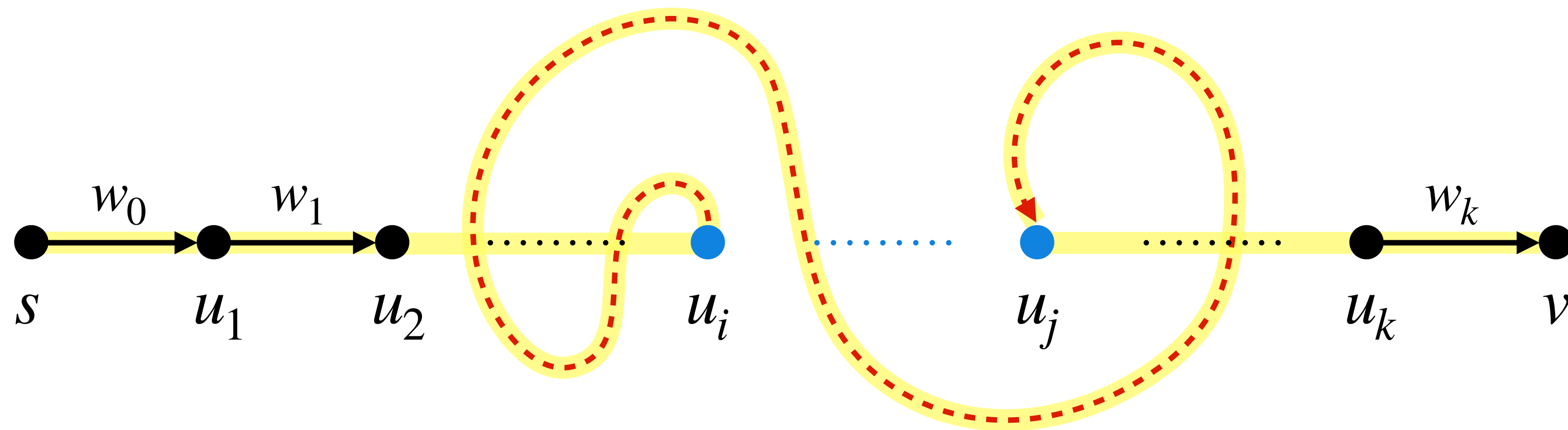
Suppose $\exists$ a shorter path from u_i to u_j .

Then, $\exists$ a walk P' from s to v with weight $< w$.

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof: Let P be a shortest path from s to v of weight w .



Suppose $\exists$ a shorter path from u_i to u_j .

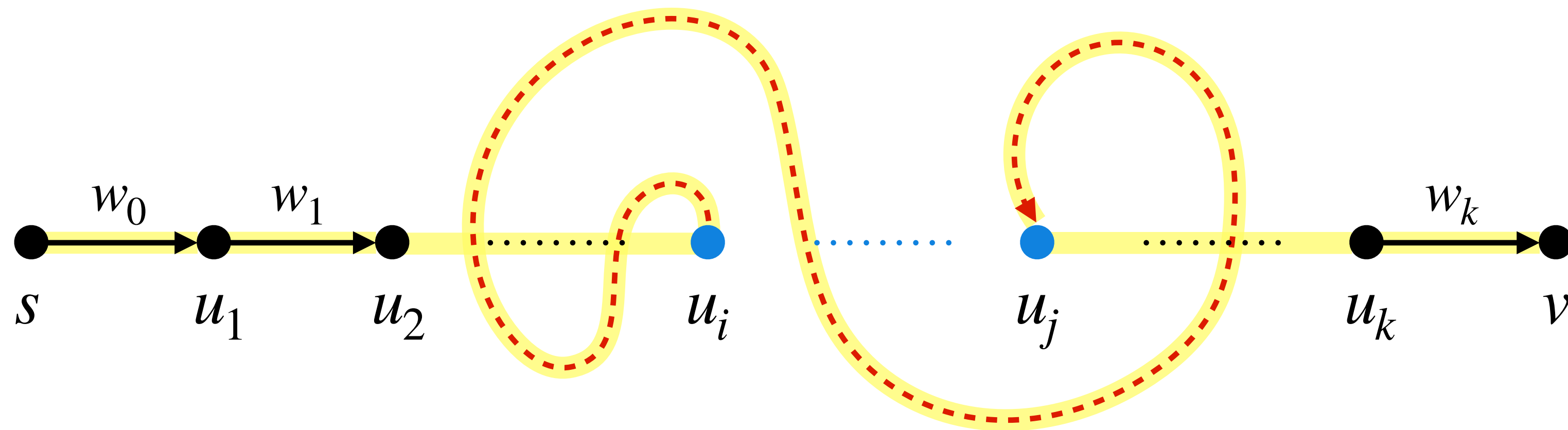
Then, $\exists$ a walk P' from s to v with weight $< w$.

Now, by removing cycles of ≥ 0 weight from P' we can get an s - v path of weight $< w$.

Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof: Let P be a shortest path from s to v of weight w .



Suppose $\exists$ a shorter path from u_i to u_j .

Then, $\exists$ a walk P' from s to v with weight $< w$.

Now, by removing cycles of ≥ 0 weight from P' we can get an s - v path of weight $< w$.

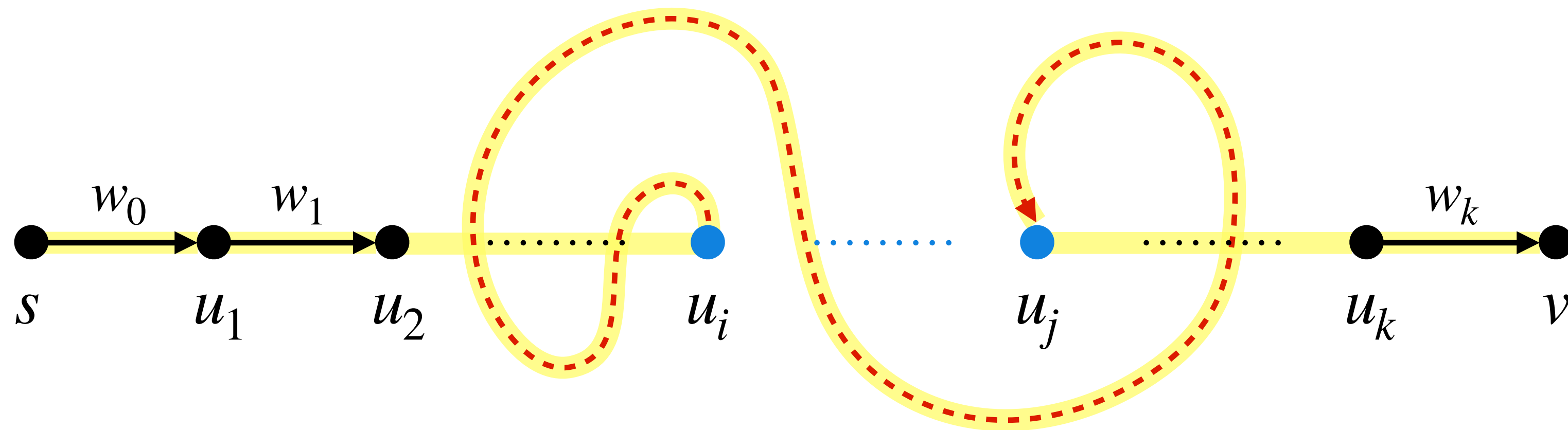
A contradiction!



Optimal Subpath Property

Lemma: In a weighted graph with **no negative weight cycles** every subpath of a shortest path is also a shortest path.

Proof: Let P be a shortest path from s to v of weight w .



Suppose $\exists$ a shorter path from u_i to u_j .

Then, $\exists$ a walk P' from s to v with weight $< w$.

Now, by removing cycles of ≥ 0 weight from P' we can get an s - v path of weight $< w$.

